

19.1 Tree Recap

Trees and Traversals

What is a Tree?

Recall that a tree consists of:

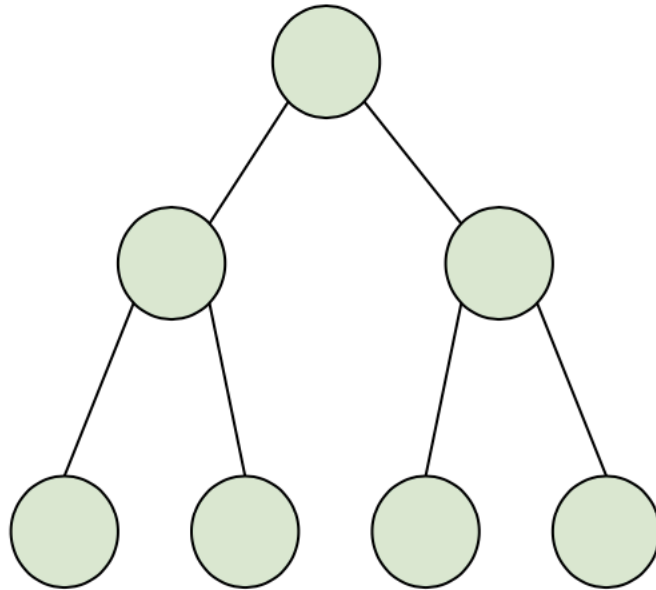
- A set of connected nodes (or vertices). We use both terms interchangeably.
- A set of edges that connect those nodes. No edges can form a cycle.
 - **Constraint:** There is exactly one path between any two nodes.

Picture Credits: Mihir Mirchandani

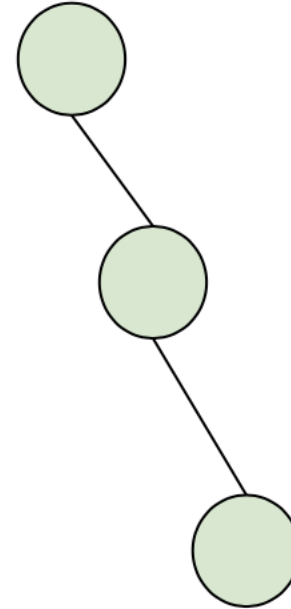
Graph #1



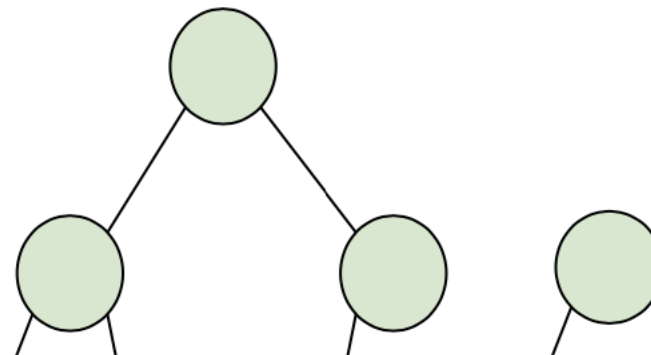
Graph #2

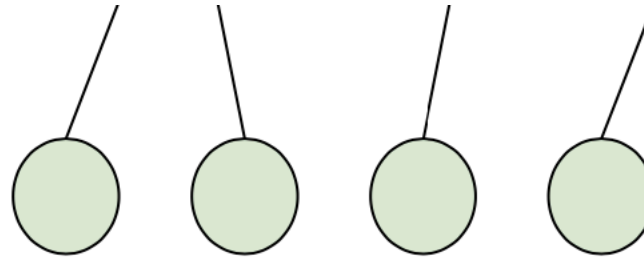


Graph #3



Graph #5





Graph Examples (Some Trees, some not...)

Graph #1 is a tree. It has a node. It has no edges. That's *OK*!

The second and third structures are trees. Notice that the third is a LinkedList. A LinkedList is still a tree as it is a connected acyclic graph.

The fourth is not a tree. Why? There are two paths from the top node to the bottom node, and so this does not obey our constraint.

Exercise 17.1.1. Determine the reason why the fifth structure is not a tree. Also, modify the invalid trees above so that they are valid.

What is a rooted tree?

Recall that a rooted tree is a tree with a designated root (typically drawn as the top most node.)

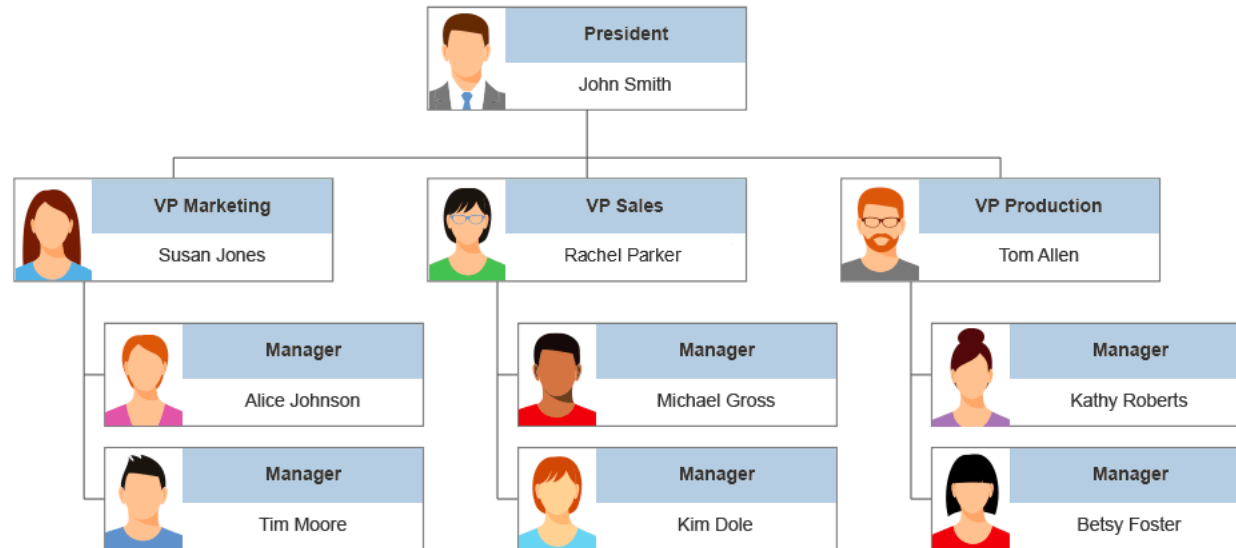
This gives us the notion of two more definitions

- A parent. Every node except the root has exactly one parent.
 - What if a node had 2 parents? Would it be a tree? (Hint: No.)
- A child. A node can have 0 or more children.
 - What if a node has 0 children? It's called a leaf.

What are trees useful for?

So far, we've looked at Search Trees, Tries, Heaps, Disjoint Sets, etc. These were extremely useful in our journey to create efficient algorithms: speeding up searching for items, allowing prefixing, checking connectedness, and so on.

But the fact of the matter is that they are even more ubiquitous than we realize. Consider an organization chart. Here, the President is the 'root'. The 'VP's are children of the root, and so on.



Organization Chart

Another tree structure is the 61b/ directory on your Desktop (it is on your Desktop, isn't it?). As we can see, when you traverse to a subfolder it goes to subsequent subfolders and so on. This is exactly tree-like!

Exercise 17.1.2. Think of other common uses of trees that weren't mentioned above. Try and determine possible implementations or designs of these trees.

19.2 Tree Traversals

Tree ~~Iteration~~ Traversal

Remember how we learned to iterate through lists? There was a way to iterate through lists that felt natural. Just start at the beginning... and keep going.

Or maybe we did some things that were a little strange, like iterate through the reverse of the list. Recall we also wrote iterators in [discussion](#) to skip over students who didn't write descriptions on the Office Hours queue.

Now how do you iterate over a tree? What's the correct 'order'?

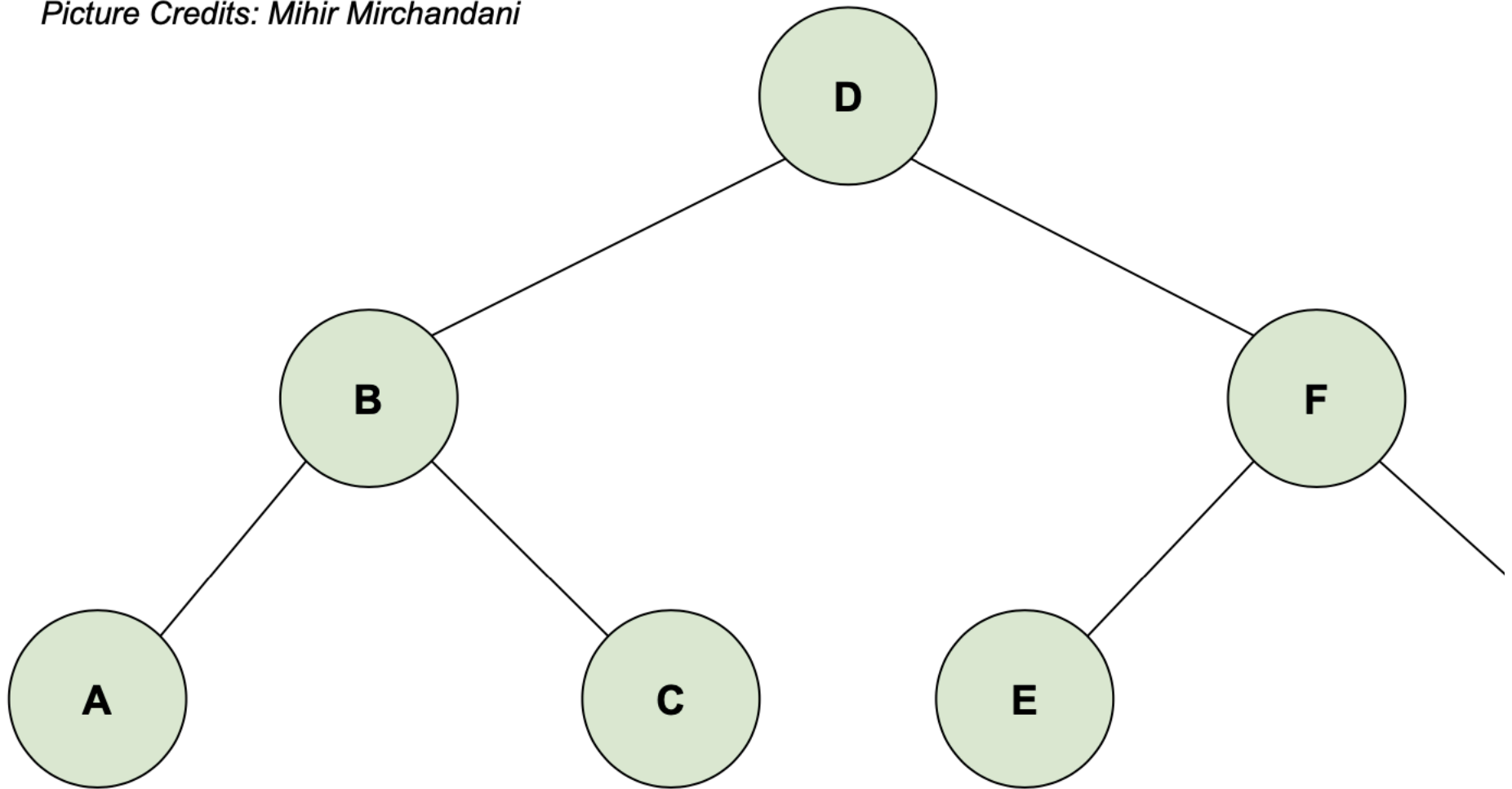
Before we answer that question, we must not use the word iteration. Instead, we'll call it 'traversing through a tree' or a 'tree traversal'. Why? No **real** reason, except that everyone calls iteration through trees 'traversals'. Maybe it's because the world likes alliterations.

So what are some natural ways to 'traverse' through a tree? As it turns out, there are a few — unlike a list which basically has one natural way to iterate through it:

1. Level order traversal. Breadth First Search (BFS)
2. Depth-First traversals — of which there are three: pre-order, in-order and post-order. (DFS)

Let's test out these traversals mentioned above on the tree below.

Picture Credits: Mihir Mirchandani



Different Traversals on One Tree...

Level Order Traversal

We'll iterate by levels, left to right. Level 0? D. Level 1? B and then F. Level 2? A, C, E, and G.

This gives us D B F A C E G.

Imagine each level was a sentence in an english book, and we just read it off line by line.

Exercise 17.2.1. Write the code for performing a level order traversal (warning, this is more difficult than the writing the other traversals). *Hint:* You will want to keep track of what level you are at.

Pre-order Traversal

Here's the idea behind pre-order traversal. Start at the root. **Visit the root** (aka, do the **action** you want to do.) The action here is "print".

So, we'll print the root. D. There we go.

Now, go left, and recurse. Then, go right and recurse.

So now we've gone left. We're at the B node. Print it. B. We'll go left after printing. (Remember, after we're done with B's left branch, we'll come back up and visit B's right.)

Keep following this logic, and you get D B A C F E G.

```
preOrder(BSTNode x) {
    if (x == null) return;
    print(x.key)
    preOrder(x.left)
    preOrder(x.right)
}
```

In-order Traversal

Slightly different, but same big-picture idea. Here, instead of **visiting** (aka **printing**) first, we'll first visit the left branch. Then we'll print. Then we'll visit the right branch.

So we start at D. We don't print. We go left.

We start at B. We don't print. We go left.

We start at A. We don't print. We go left. We find nothing. We come back up, and print A.

Then go to A's right. Find nothing. Go back up. Now we're at B. Remember, we print after visiting left and before visiting right, so now we'll print B, then we'll visit right.

Keep following this and you get A B C D E F G.

An alternative way to think about this is as follows:

First, we're at D. We know we'll print out the items from left, then D, then items from right.

[items from left] D [items from right].

Now what's [items from left] equal to? We'll start at B, print out left, then print B, then print stuff from right of B.

[items from left] = [items from B's left] B [items from B's right] = A B C.

A B C D [stuff from right] = A B C D E F G.

```
inOrder(BSTNode x) {
    if (x == null) return;
    inOrder(x.left)
    print(x.key)
    inOrder(x.right)
}
```

Post-order Traversal

Again, same big-picture idea, but now we'll print left branch, then right branch, then ourselves.

Using the method we devised earlier, the result looks like:

[items from left] [items from right] D.

What's [items from left]? It's the output from the B subtree.

If we're at B, we'd get [items from left of B] [items from right of B] B, which is equal to A C B.

Following this through, we get: A C B E G F D

```
postOrder(BSTNode x) {
    if (x == null) return;
    postOrder(x.left);
    postOrder(x.right);
    print(x.key);
}
```

19.3 Graphs

Trees are great, aren't they? But as we saw, we could draw some things using nodes and edges that weren't trees. Specifically, our restriction that there can only be one path between any two nodes didn't fit every situation. Let's see what happens when we get rid of that restriction.

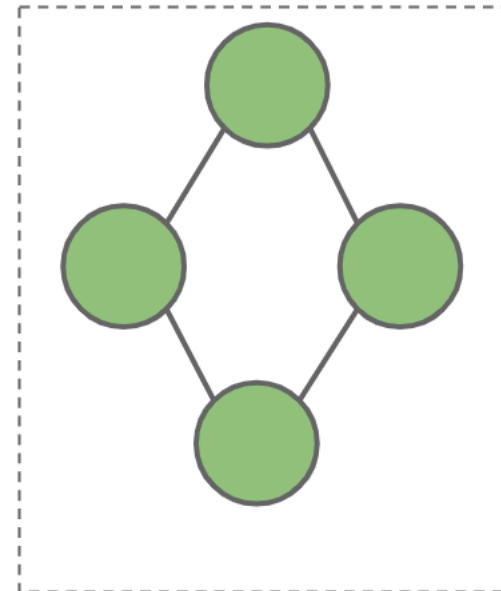
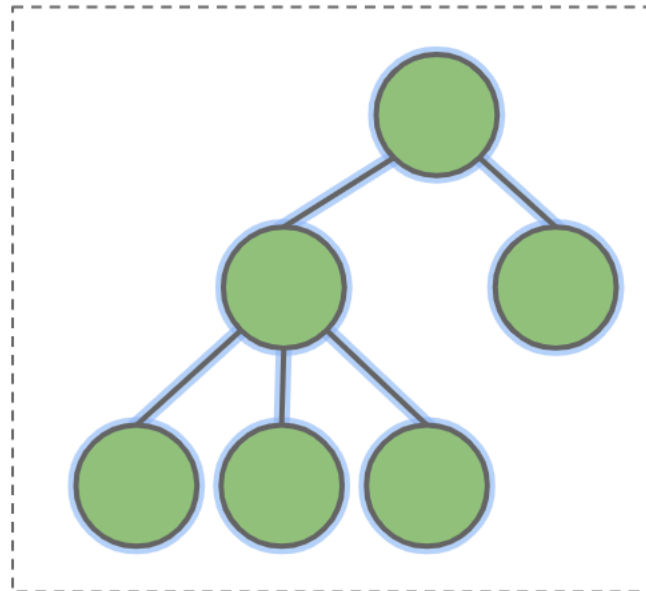
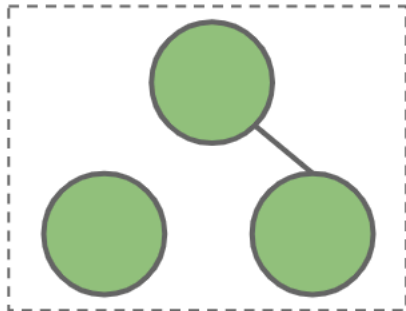
What is a graph?

A graph consists of:

- A set of nodes (or vertices)
- A set of zero or more edges, each of which connects two nodes.

That's it! No other restrictions.

All of the structures below in green? Everything is a valid graph! The second one is also a tree, but none of the others are.



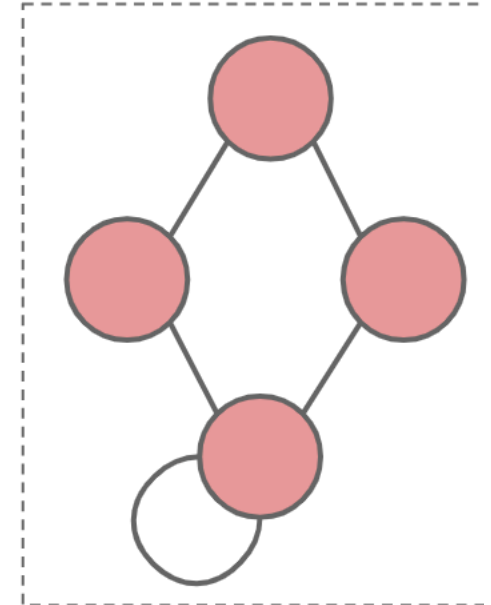
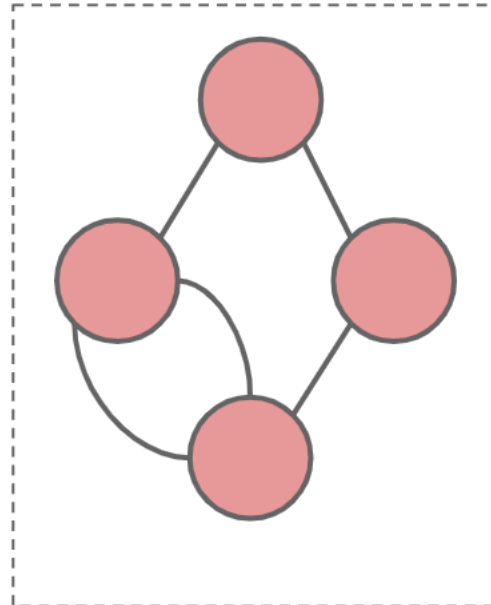
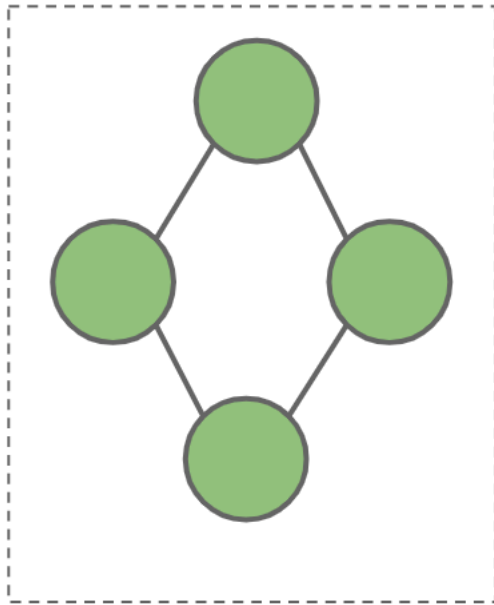
Graphs

In general, note that **all trees are also graphs, but not all graphs are trees.**

Simple Graphs only

Graphs can be divided into two categories: *simple* graphs and *multigraphs* (or complicated graphs, a term I invented, because that's how I like to think of them.) Fortunately, in this course (and almost all applications and research) focuses only on simple graphs. So when we say “graph” in this course, you should always think of a “simple graph” (unless we say otherwise.)

Well, it's time to address the elephant in the room. What's a simple graph?



Simple Graphs and Multigraphs

Look at the graphs in red. The graph in the middle has 2 distinct edges going from/to bottom-next to/from bottom-left node. In other words, there are multiple edges between two nodes. This is **not** a simple graph, and we ignore their existence unless specified otherwise. Graphs like these are called multigraphs.

Look at the third graph. It has a loop! An edge from a node to itself! We don't allow this either. Graphs like these are sometimes categorized as multigraphs, and sometimes, even multigraphs explicitly ban self-loops.

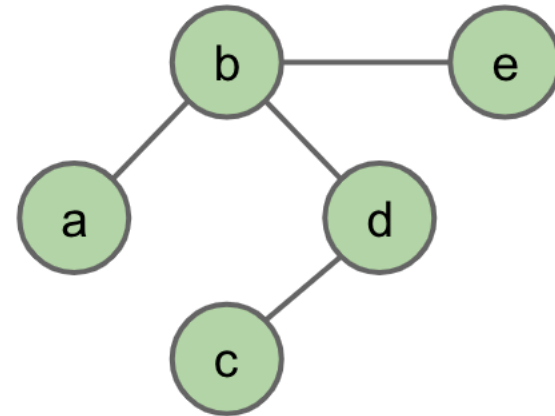
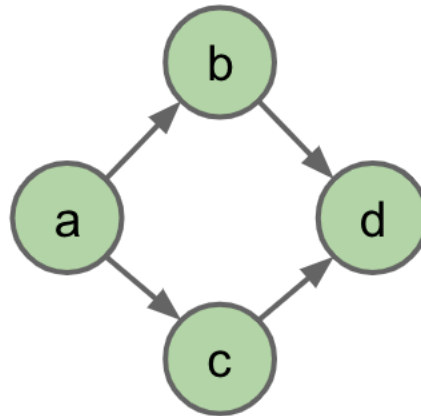
More categorizations.

Graphs are simple in the following text, and in this course, unless specified otherwise. But there are more categorizations.

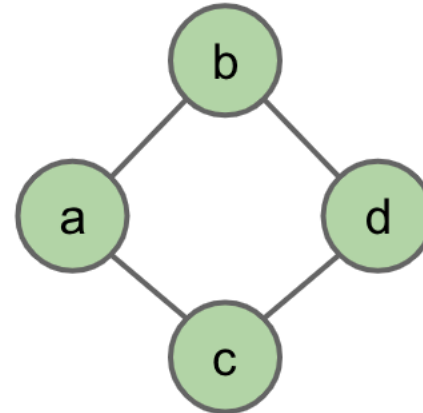
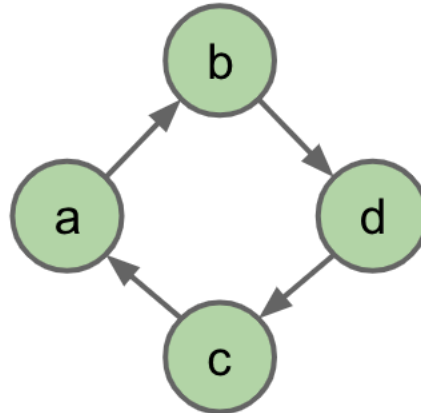
Directed

Undirected

Acyclic:



Cyclic:



There are undirected graphs, where an edge (u, v) can mean that the edge goes from the nodes u to v and from the nodes v to u too. There are directed graphs, where the edge (u, v) means that the edge starts at u , and goes to v (and the vice versa is not true, unless the edge (v, u) also exists.) Edges in directed graphs have an arrow.

There are also acyclic graphs. These are graphs that don't have any cycles. And then there are cyclic graphs, i.e., there exists a way to start at a node, follow some **unique** edges, and return back to the same node you started from.

In the above picture, we can clearly see the difference between how we draw directed and undirected edges.

Take a look at the cyclic graphs. If you start at a , you can run back around using only distinct edges and get back to a . Thus, the graph is cyclic.

Take a look at the top-left graph. Is there any node n , such that if you start at n , you can follow some distinct edges, and get back to n ? Nope! (Remember than for directed edges, you must follow the directions. You can go from a to b but not b to a .)

19.4 Graph Problems

There are many questions we can ask about a graph.

For example,

- **s-t Path:** Is there a path between vertices s and t ?
- **Connectivity:** Is the graph connected, i.e. is there a path between all vertices?
- **Biconnectivity:** Is there a vertex whose removal disconnects the graph?
- **Shortest s-t Path:** What is the shortest path between vertices s and t ?
- **Cycle Detection:** Does the graph contain any cycles?
- **Euler Tour:** Is there a cycle that uses every edge exactly once?
- **Hamilton Tour:** Is there a cycle that uses every vertex exactly once?
- **Planarity:** Can you draw the graph on paper with no crossing edges?
- **Isomorphism:** Are two graphs isomorphic (the same graph in disguise)?

What's cool and also weird about graph problems is that it's very hard to *tell* which problems are very hard, and which ones aren't all that hard.

For example, consider the Euler Tour and the Hamilton Tour problems. The former... is a solved problem. It was solved as early as 1873. The solution runs in $O(E)$ where E is the number of edges in the graph.

The latter? If you were to solve it efficiently today, you would win every Math award there was, become one of the most famous computer scientists, win a million dollars, etc. No one has been able to solve this **efficiently**. The best known algorithms run in exponential times. People have been working on it for many decades!

One step at a time!

Alright, well, before we solve the million dollar problem, let's solve the first one on the list. Given a source vertex s and a target vertex t , is there a path between s and t ?

In other words, write a function `connected(s, t)` that takes in two vertices and returns whether there exists a path between the two.

To begin, let's guess that we have the following code:

```
if (s == t):
    return true;

for child in neighbors(s):
    if isconnected(child, t):
        return true;

return false;
```

Exercise 17.4.1. Before you move on, please read the code above, and spend time thinking about it. Does it work? Is it efficient? Run through a couple scenarios.

Alright, so, let's try it out.

We start with `connected(0, 7)`? That recursively calls `connected(1, 7)`, which then recursively calls `connected(0, 7)`. Uh-oh. Infinite looping has occurred.

Exercise 17.4.2. How could we fix this? Once again, thinking about this. What was the problem? We visited s again... but did we need to?

Alright, let's try a "remember what we visited" approach.

```
mark s // i.e., remember that you visited s already
if (s == t):
    return true;

for child in unmarked_neighbors(s): // if a neighbor is marked, ignore!
    if isconnected(child, t):
        return true;

return false;
```

As it turns out, this does work! Follow the example in [these slides](#) to see how.

Woah, what did we just develop?

You may not have realized it, but we just developed a **depth-first traversal** (like pre-order, post-order, in-order) but for graphs. What did we do? Well, we marked ourself. Then we visited our first child. Then our first child marked itself, and visited its children. Then our first child's first child marked itself, and visited its children.

Intuitively, we're going deep (i.e., down our family tree to our first child, our first child's first child aka our first grandchild, our first grandchild's first child, and so on... visiting this entire lineage), before we even touch our second child.

Up next, we'll see the opposite notion, where first we visit all our children, then our grandchildren, and so on.

19. Tree Traversals and Graphs